

Day 120 — Final Beginner Python, Data Science and Machine Learning Mini Project

1. Day Number

Day 120 of 120 🏆

You have reached the final day of the beginner journey.

Today is not about learning a large new topic. It is about connecting the skills you have already learned into one small end-to-end application.

2. Topic

Simple End-to-End ML Model Exposed Through FastAPI

Today you will build the conceptual structure for a small application that:

loads data → cleans data → trains a model → evaluates it → makes predictions → exposes predictions through an API

The project will intentionally stay small.

You are **not** trying to build a production ML platform.

3. Connection to Days 1–120

Throughout the journey, you gradually moved from basic Python programs toward data science and machine learning.

You learned ideas from:

- Python fundamentals
- functions and modules
- error handling
- files and data formats
- virtual environments
- Git
- CSV, JSON, and APIs
- NumPy
- Pandas
- data cleaning
- visualization and EDA
- SQL and databases
- basic mathematics
- statistics and probability
- machine-learning foundations

- supervised learning
- scikit-learn
- model evaluation
- FastAPI

Day 120 combines these pieces into one workflow.

The important idea is:

| A machine-learning model by itself is only one part of an ML application.

We also need to understand the data before training, evaluate the model after training, and provide some way for another program to request predictions.

4. Revision Summary — Major Skills from Days 61–119

Developer and project skills

You learned how Python projects can be organized rather than keeping everything in one giant file.

You worked with ideas such as:

project folder
virtual environment
requirements
Git repository
Python modules

You also learned why reproducible environments matter.

For example:

`requirements.txt`

can record packages such as:

pandas
scikit-learn
fastapi
uvicorn

Data collection and formats

You worked with common data formats:

CSV
Excel
JSON
API responses

You learned that real-world data often comes from outside your Python program.

External data can also fail because of:

- missing files
- bad JSON
- incorrect values
- network problems

NumPy

You learned the idea of numerical arrays.

For example:

```
[1200, 1500, 1800, 2100]
```

You practiced:

- indexing
- slicing
- sum
- mean
- basic arithmetic

NumPy forms part of the numerical foundation underneath many data-science libraries.

Pandas

You learned to work with structured tabular data using a DataFrame.

Example:

size	bedrooms	price
900	2	250000
1200	3	320000
1500	3	390000

You learned operations such as:

- head()
- shape
- columns
- dtypes
- isna()
- fillna()
- dropna()
- drop_duplicates()

You also learned to inspect data **before** changing it.

Data cleaning

You learned to look for:

missing values
duplicates
wrong data types
invalid values

A major lesson was:

| Cleaning decisions should depend on what the data actually represents.

We should not automatically remove every row containing a missing value.

Visualization and EDA

You used tools such as:

Matplotlib
Seaborn

and learned basic exploratory data analysis.

You inspected:

dataset shape
column types
missing values
summary statistics
distributions
relationships

EDA helps answer an important question before ML:

| Does this dataset make sense?

Databases

You were introduced to different ways of storing data, including:

SQL databases
MongoDB
Redis

You learned SQL ideas such as:

SELECT
WHERE
JOIN

A real ML application may eventually obtain its data from a database instead of a CSV file.

For this final beginner project, however, a CSV is enough.

Mathematics

You studied foundational ideas such as:

- algebra
- vectors
- matrices
- mean
- median
- variance
- standard deviation
- probability
- distributions
- derivatives
- gradients
- gradient descent

You do not need to manually calculate all this mathematics when using scikit-learn.

But understanding the concepts helps explain what machine-learning algorithms are doing.

Machine-learning foundations

You learned important terms:

Feature

Information used by the model to make a prediction.

Label / target

The value the model is trying to predict.

You also learned:

- training data
- validation idea
- test data
- overfitting
- underfitting
- bias
- variance
- feature engineering

Supervised machine learning

You explored models including:

- Linear Regression
- Logistic Regression
- K-Nearest Neighbors
- Naive Bayes

Decision Tree
Random Forest

You also learned that the correct evaluation metric depends on the problem.

Classification examples:

accuracy
precision
recall
F1
confusion matrix

Regression examples:

MAE
MSE
RMSE

FastAPI

Finally, you learned how Python code can expose functionality through an HTTP API.

Instead of manually running:

```
predict(...)
```

another application could send a request such as:

```
{  
  "size": 1200,  
  "bedrooms": 3  
}
```

and receive:

```
{  
  "prediction": 325000  
}
```

That completes the connection between a trained model and an external user/program.

5. Important Topics for Today's Project

Project structure

Avoid putting the entire project into one giant file.

A small beginner structure might look like:

```
final_ml_project/  
|  
├─ data/  
|   └─ houses.csv
```

```
|  
├─ train.py  
├─ prediction.py  
├─ main.py  
├─ requirements.txt  
└─ README.md
```

Conceptually:

`train.py`

handles model training.

`prediction.py`

handles prediction logic.

`main.py`

contains the FastAPI application.

You could simplify the structure further while learning. The purpose is simply to separate responsibilities.

Virtual environment

A virtual environment isolates the project's Python packages.

Conceptually:

`final_ml_project`

↓

virtual environment

↓

install only required libraries

Typical packages might include:

`pandas`

`scikit-learn`

`fastapi`

`uvicorn`

Requirements

A `requirements.txt` file records dependencies.

Conceptually:

`pandas`

`scikit-learn`

`fastapi`

`uvicorn`

This makes it easier to recreate the environment later.

Pandas

Pandas loads and inspects your dataset.

Conceptually:

```
data = read CSV
```

```
inspect first rows  
inspect shape  
inspect missing values
```

Do not immediately start training.

First understand the dataset.

Basic data cleaning

Only perform cleaning that is actually necessary.

For example:

```
remove duplicate rows  
handle one missing value  
convert numeric text into numeric values
```

Avoid turning the final project into a huge cleaning exercise.

Features and label

Suppose the dataset predicts house price.

Dataset:

size	bedrooms	age	price
900	2	10	250000
1200	3	5	320000
1500	3	2	390000

Features:

```
size  
bedrooms  
age
```

Target:

```
price
```

Conceptually:

X = input columns

y = target column

Think:

$X \rightarrow$ information given to model

$y \rightarrow$ answer model should learn

6. Foundational Notes

ML is learning a relationship

Suppose you have:

size $\rightarrow$ price

A regression model tries to learn a useful relationship between those values.

With several features:

```
size
bedrooms
age
  ↓
model
  ↓
price
```

Training is not the same as testing

The model learns using training data.

Then you evaluate it on data it did **not** train on.

Conceptually:

```
Entire dataset
  ↓
train_test_split
  ↙   ↘
train  test
  ↓   ↓
learn  evaluate
```

Without this separation, you could get an overly optimistic picture of the model.

Start with one simple model

Do not compare fifteen algorithms today.

For example, use:

Linear Regression

for predicting a numerical value.

Or use:

Logistic Regression

for a simple two-class classification problem.

The point is understanding the complete workflow.

Evaluation happens before API exposure

You should not immediately put a model behind FastAPI just because:

```
model.fit(...)
```

ran successfully.

First ask:

Did the model perform reasonably on unseen test data?

For Linear Regression you might calculate:

MAE

RMSE

For Logistic Regression you might calculate:

accuracy

F1

For the beginner project, **one or two metrics are enough**.

A prediction function separates ML from the API

Instead of placing all model logic inside the endpoint, think of:

prediction function

as an intermediate layer.

Conceptually:

API request

↓

prediction function

↓

trained model

↓

prediction

This separation makes the program easier to understand.

FastAPI does not train the model conceptually

Think of two stages.

Training stage

dataset

↓

train model

↓

evaluate model

Prediction stage

new user input

↓

trained model

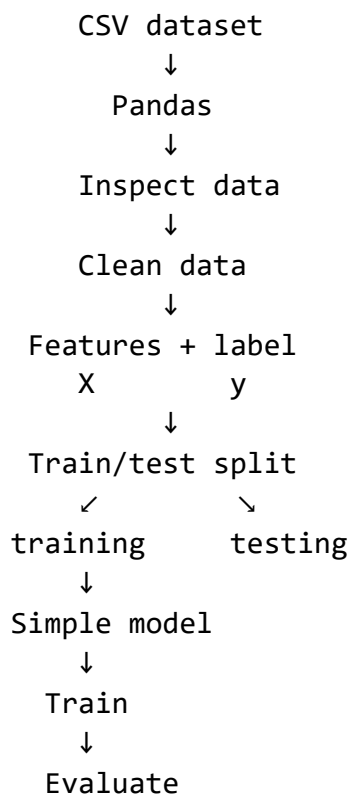
↓

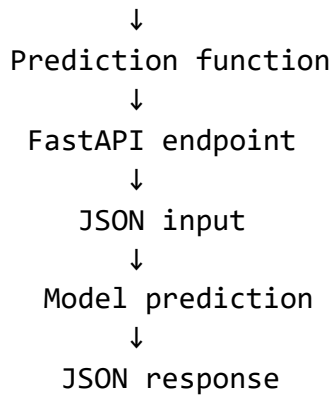
prediction

The API belongs mainly to the **prediction stage**.

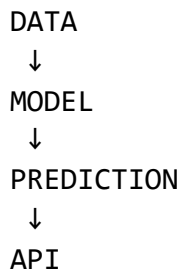
7. Easy Architecture Diagram

Your complete beginner workflow is:





An even simpler mental model is:



Those four stages summarize today's project.

8. Final Problem Statement

Build a Very Small ML Prediction Application

Suppose you have a tiny house-price dataset.

Example:

`houses.csv`

size	bedrooms	age	price
800	2	12	210000
1000	2	8	260000
1200	3	6	315000
1400	3	4	360000
1600	4	2	430000

Your target is:

`price`

Your features could be:

`size`
`bedrooms`
`age`

Your model could be:

LinearRegression

Your program should:

1. Load the CSV.
2. Inspect the dataset.
3. Perform only necessary cleaning.
4. Separate:
 - X = features
 - y = target
5. Split the data into:
 - training data
 - testing data
6. Create one Linear Regression model.
7. Train it.
8. Make predictions on the test data.
9. Calculate one or two regression metrics.
10. Create a function that accepts:
 - size
 - bedrooms
 - age
11. Use the trained model to predict price.
12. Create one FastAPI endpoint.
13. Receive input values as JSON.
14. Pass those values to the prediction function.
15. Return the predicted price as JSON.

Example request:

```
{
  "size": 1300,
  "bedrooms": 3,
  "age": 5
}
```

Conceptual response:

```
{  
  "predicted_price": 340000  
}
```

The exact predicted value is not important for the exercise.

Understanding the flow is.

9. Concepts Used

This small project combines many earlier topics:

Concept	Purpose
Python	overall programming
functions	organize prediction logic
modules	separate files
virtual environment	isolate dependencies
requirements	record packages
CSV	store sample data
Pandas	load and clean data
DataFrame	represent table data
features	model inputs
target	value being predicted
train/test split	separate learning and evaluation
Linear Regression	simple regression model
MAE/RMSE	evaluate prediction error
scikit-learn	ML tools
error handling	deal with failures
FastAPI	expose predictions
endpoint	URL receiving requests
JSON	send input and output

Notice how little completely new material exists today.

The main challenge is **connecting the pieces correctly**.

10. Thought Process — Raw Data to API Prediction

Think through the project in this order.

Step 1 — What am I predicting?

Before writing ML code, define the question.

Example:

| Predict a house's price from its size, bedrooms, and age.

Therefore:

problem type = regression

because price is numerical.

Step 2 — What data do I have?

Inspect:

rows
columns
data types
missing values
duplicates

Ask:

Does every column mean what I think it means?

Step 3 — Does anything need cleaning?

Perhaps:

one missing value
one duplicate
one numeric column stored incorrectly

Fix only those problems.

Do not invent complicated cleaning requirements.

Step 4 — Which columns are features?

Example:

size
bedrooms
age

These become:

Step 5 — Which column is the label?

Example:

price

This becomes:

y

Step 6 — How will I test the model?

Split your data.

For example, conceptually:

80% → training

20% → testing

The exact split is less important than understanding why both sets exist.

Step 7 — Which model fits the task?

Because you are predicting a continuous number:

Linear Regression

is a sensible beginner choice.

Step 8 — Train

Conceptually:

```
model.fit(training_features, training_labels)
```

The model learns patterns from the training data.

Step 9 — Evaluate

Use:

test features

to produce predictions.

Then compare:

actual prices

vs

predicted prices

For example, calculate:

MAE

and optionally:

RMSE

Step 10 — Create reusable prediction logic

Your prediction function might conceptually behave like:

receive:

```
size
bedrooms
age
```

prepare values in same format as training data

send values to model

receive predicted price

return prediction

Step 11 — Create the API

FastAPI receives JSON such as:

```
{
  "size": 1250,
  "bedrooms": 3,
  "age": 4
}
```

FastAPI then calls:

prediction function

Step 12 — Return JSON

The API response could conceptually be:

```
{
  "predicted_price": 335000
}
```

You have now connected:

HTTP request
↓
Python
↓
machine learning
↓
HTTP response

That is the key achievement of Day 120.

11. Beginner-Friendly Pseudocode

This is deliberately **pseudocode**, not the complete final Python solution.

START PROGRAM

SET UP PROJECT

create virtual environment

install:
 pandas
 scikit-learn
 fastapi
 uvicorn

record dependencies

TRAINING SECTION

IMPORT required libraries

LOAD CSV using Pandas

DISPLAY first few rows

CHECK:
 shape
 columns
 data types
 missing values
 duplicates

IF simple cleaning is required:
 clean the problem

SELECT feature columns:

size
bedrooms
age

SELECT target column:

price

STORE features as X

STORE target as y

SPLIT X and y into:

X_train
X_test
y_train
y_test

CREATE Linear Regression model

TRAIN model using:

X_train
y_train

PREDICT using:

X_test

CALCULATE:

MAE

OPTIONALLY calculate:

RMSE

PRINT evaluation results

PREDICTION SECTION

CREATE function predict_price(size, bedrooms, age):

convert inputs into correct model input shape

```
    call model prediction
```

```
    get first prediction
```

```
    return prediction
```

```
-----  
FASTAPI SECTION  
-----
```

```
CREATE FastAPI application
```

```
DEFINE structure of JSON input:
```

```
    size
```

```
    bedrooms
```

```
    age
```

```
CREATE prediction endpoint
```

```
WHEN request arrives:
```

```
    receive input
```

```
    validate input
```

```
    call predict_price()
```

```
    IF prediction succeeds:
```

```
        return JSON containing predicted price
```

```
    ELSE:
```

```
        return useful error
```

```
START API server
```

```
END
```

The most important part of this pseudocode is the sequence:

```
load
```

```
↓
```

```
clean
```

```
↓
```

```
split
```

```
↓
```

```
train
```

↓
evaluate
↓
predict
↓
serve

12. Suggested Solving Approach

Use scikit-learn + FastAPI

Build the project in small stages instead of trying to create everything at once.

Stage A — Make the data code work

First achieve:

CSV
↓
Pandas DataFrame

Check the data manually.

Stage B — Make the ML workflow work

Next achieve:

DataFrame
↓
X + y
↓
train/test
↓
model
↓
metric

At this point, do **not** worry about FastAPI.

Confirm that your model can actually make predictions.

Stage C — Create the prediction function

Next achieve:

predict_price(1200, 3, 5)
↓
prediction

Again, no API yet.

Stage D — Create FastAPI

Once prediction works:

```
POST request
  ↓
FastAPI
  ↓
prediction function
  ↓
JSON response
```

This makes debugging much easier.

If something fails, you can determine whether the problem belongs to:

```
data
ML
prediction function
API
```

instead of debugging everything simultaneously.

13. Easy Edge Cases

Edge Case 1 — Missing input

Suppose the endpoint expects:

```
{
  "size": 1200,
  "bedrooms": 3,
  "age": 5
}
```

but receives:

```
{
  "size": 1200,
  "bedrooms": 3
}
```

age is missing.

Your API should not silently guess an age unless that behavior was deliberately designed.

FastAPI/Pydantic validation can help identify the missing field.

Edge Case 2 — Invalid numeric input

Expected:

```
{  
  "size": 1200  
}
```

But the client sends:

```
{  
  "size": "very large"  
}
```

That cannot sensibly be used as a numerical model feature.

Input validation should catch invalid values before they reach the model.

Also think about logically impossible values:

```
size = -500  
bedrooms = -2
```

Something being a valid Python number does not automatically make it valid real-world data.

Edge Case 3 — Unexpected category

Your simplest house example uses only numerical features, so this issue may not occur.

But imagine adding:

```
location_type
```

with training values:

```
urban  
suburban  
rural
```

A request might later contain:

```
underwater_city
```

If your preprocessing does not know how to handle that category, prediction can fail.

The broader lesson is:

| New API input must be transformed in exactly the way your training data was transformed.

For Day 120, keeping all features numerical is perfectly reasonable.

Edge Case 4 — Model cannot make prediction

A prediction could fail because:

```
input shape is incorrect  
wrong data type reached the model  
required preprocessing was skipped
```

```
model is unavailable
unexpected program error occurred
```

Your API should avoid exposing a huge confusing Python traceback to the client.

Conceptually:

```
try prediction
```

```
if successful:
    return prediction
```

```
if failure:
    return simple error response
```

Keep error handling small for this project.

14. Common Mistakes to Avoid

Mistake 1: Training using the entire dataset

If you train using every row and then evaluate using those same rows, you do not have a proper unseen test set.

Use:

```
train
+
test
```

Mistake 2: Accidentally including the target inside the features

Wrong concept:

```
features =
size
bedrooms
age
price
```

while also predicting:

```
price
```

The answer must not be included as an input feature.

Correct:

```
X:
size
bedrooms
age
```

y:
price

Mistake 3: Cleaning before inspecting

Do not immediately run operations such as:

drop everything missing
remove columns
convert everything

First inspect the data.

Mistake 4: Choosing the wrong model type

If predicting:

price = 325000

you have a regression problem.

If predicting:

spam / not spam

you have a classification problem.

Mistake 5: Choosing the wrong metric

For price prediction, something like:

MAE
RMSE

makes sense.

Plain classification accuracy does not.

Mistake 6: Evaluating using training predictions only

The important evaluation is how the model behaves on data that was not used for training.

Mistake 7: Different feature order during prediction

Imagine training with:

size
bedrooms
age

and accidentally predicting with:

age
size

bedrooms

Those values may all be valid numbers, but they represent completely different things.

Keep input structure consistent.

Mistake 8: Training and API using different preprocessing

Suppose training converts categories into numeric values.

Then the API must apply the same transformation.

Conceptually:

training:

```
raw data
  ↓
transformation
  ↓
model
```

API:

```
new input
  ↓
SAME transformation
  ↓
model
```

Mistake 9: Building FastAPI before the model works

Verify this first:

```
Python function
  ↓
prediction
```

Then add:

FastAPI

Mistake 10: Making the final project too large

You do **not** need:

- 50 features
- millions of rows
- many models
- complex frontend
- multiple endpoints

cloud deployment
Docker
Kubernetes
authentication
deep learning

A tiny working flow demonstrates the concepts more clearly.

15. Five Quick Self-Check Questions

Question 1

Suppose your dataset contains:

age
salary
experience
approved

and you want to predict approved.

Which columns are likely the features, and which is the target?

Question 2

Why should the model be evaluated using test data rather than only the training data?

Question 3

You are predicting house prices.

Which is more appropriate?

MAE

or

classification accuracy

Why?

Question 4

What path does this request follow?

```
{  
  "size": 1400,  
  "bedrooms": 3,  
  "age": 6  
}
```

Try filling in:

JSON request







ML model



Question 5

Why should the feature preparation used by your API be consistent with the feature preparation used while training the model?

16. Hint Only

Build the project **from the inside outward**.

Do not start with FastAPI.

Think:

Can I load the data?



Can I identify X and y?



Can I split the data?



Can I train the model?



Can I evaluate it?



Can one Python function make a prediction?



Can FastAPI call that function?

For the simplest version, choose a tiny numerical regression dataset:

size
bedrooms
age
price

with:

X = size + bedrooms + age

y = price

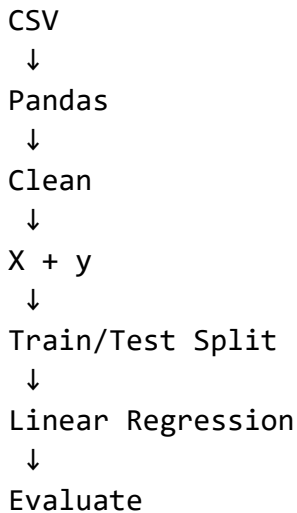
and use:

Linear Regression

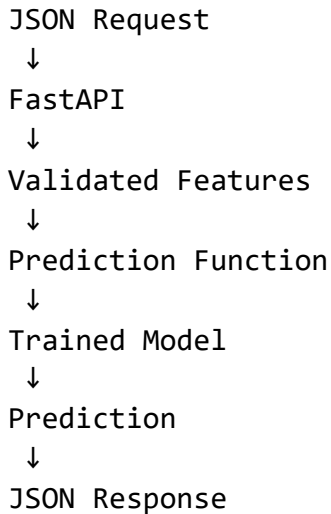
Then expose only **one POST endpoint** that accepts the three feature values and returns one prediction.

Your final mental picture should be:

LEARNING PHASE



USAGE PHASE



That is enough for Day 120.

The goal is not a production ML system. The goal is being able to explain **why every arrow in that diagram exists** and implement each part yourself.